

RENSSELAER MECHATRONICS

Blinking the LED

Lab Objectives

- Verify Arduino software package for Simulink using a digital output to light a LED
- Communicate with the target board (Arduino) using external mode by changing the brightness of an LED with PWM
- Use overrun detection to determine how fast you can run your code

Table of Contents

Lab Objectives	1
Part 1: Arduino Toolbox Installation for Simulink.....	2
Objective:	2
Simulink Arduino Library Installation:.....	2
How do I find the COM port for my Arduino Board:.....	Error! Bookmark not defined.
Part 2: Communicating with External Mode.....	5
Objective	5
Simulink.....	5
Deploy to Hardware:.....	10
External Mode – communicating with the hardware	10
External Mode:.....	11
External Mode.....	12
Stopping the Simulation:.....	13
Debugging – cannot connect to device or cannot download code	Error! Bookmark not defined.
Checkpoint:	13
Part 3 (Optional): Evaluating code execution speed – how fast can I run my code	13
Objective:	13
Simulink:.....	13
Part 4: Troubleshooting	Error! Bookmark not defined.
An error appears when trying to run the file similar to “build procedure aborted”, “make command returned an error...”	Error! Bookmark not defined.
Error appears when trying to open the Simulink file:.....	Error! Bookmark not defined.

Simulink Support for Arduino Hardware does not appear in the Simulink library browser:..... **Error! Bookmark not defined.**

Device is not recognized in device manager when it is plugged in..... **Error! Bookmark not defined.**

File is not compiling, a long list of errors appears. **Error! Bookmark not defined.**

In addition to the above, make sure that MATLAB working directory is not Windows/system32 or some other system folder that requires admin rights to edit. To be safe, set the working folder to your documents/MATLAB..... **Error! Bookmark not defined.**

Red block in Simulink diagram:..... **Error! Bookmark not defined.**

Error about board not connected to computer **Error! Bookmark not defined.**

Error Dialog. Error using open... Simulink diagram file name has parenthesis in it. **Error! Bookmark not defined.**

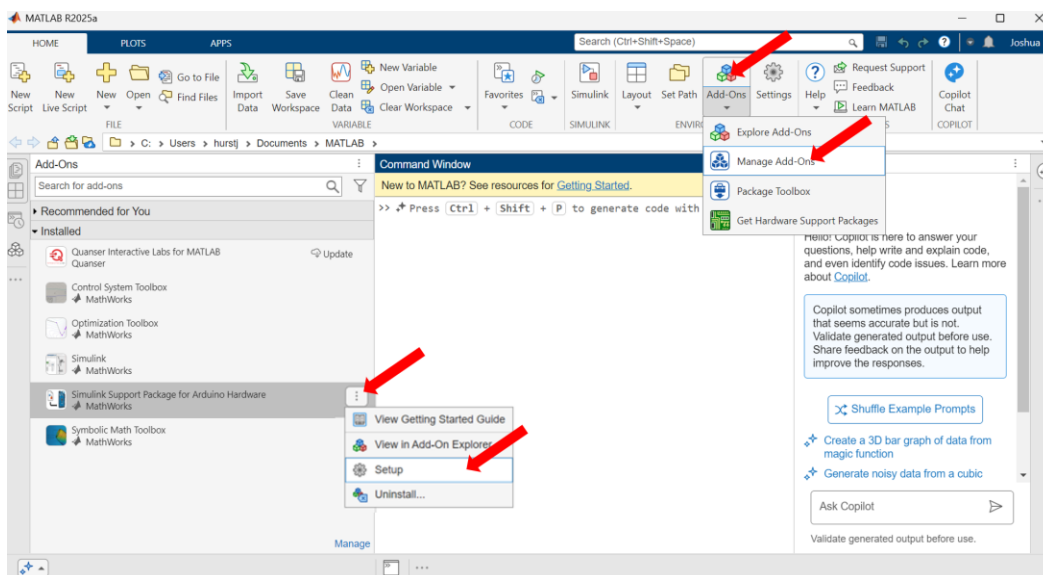
Part 1: Arduino Toolbox Installation for Simulink

Objective:

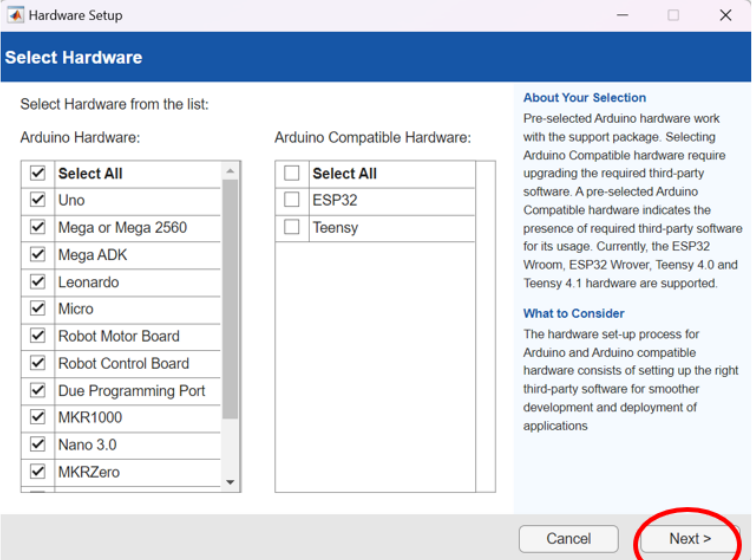
- Install and verify Arduino software package for Simulink using a digital output to light a LED with MATLAB/Simulink 2025a or later.

Verify Simulink Arduino Library Installation:

- Open MATLAB 2025a. From the MATLAB HOME window select “Add-Ons”, “Manage Add-Ons”, and click the gear icon for the Simulink Support Package for Arduino Hardware.



- Leave the default boards selected and select “Next” for the following 3 screens:



The screenshot shows the 'Hardware Setup' window with the 'Select Hardware' tab. It features two columns of hardware options. The 'Arduino Hardware' column has a 'Select All' checkbox and a list of boards, all of which are checked. The 'Arduino Compatible Hardware' column has a 'Select All' checkbox and a list of boards (ESP32, Teensy) which are unchecked. On the right, there is explanatory text under 'About Your Selection' and 'What to Consider'. At the bottom right, the 'Next >' button is circled in red.

Hardware Setup

Select Hardware

Select Hardware from the list:

Arduino Hardware:

☒	Select All
☒	Uno
☒	Mega or Mega 2560
☒	Mega ADK
☒	Leonardo
☒	Micro
☒	Robot Motor Board
☒	Robot Control Board
☒	Due Programming Port
☒	MKR1000
☒	Nano 3.0
☒	MKRZero

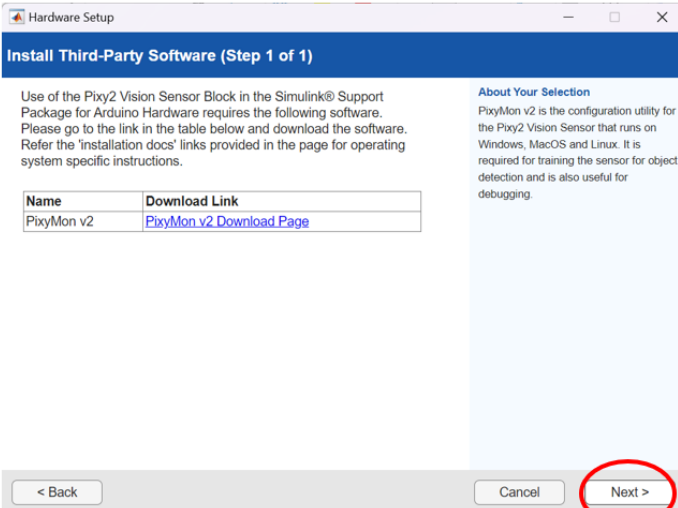
Arduino Compatible Hardware:

☐	Select All
☐	ESP32
☐	Teensy

About Your Selection
Pre-selected Arduino hardware work with the support package. Selecting Arduino Compatible hardware require upgrading the required third-party software. A pre-selected Arduino Compatible hardware indicates the presence of required third-party software for its usage. Currently, the ESP32 Wroom, ESP32 Wrover, Teensy 4.0 and Teensy 4.1 hardware are supported.

What to Consider
The hardware set-up process for Arduino and Arduino compatible hardware consists of setting up the right third-party software for smoother development and deployment of applications

Cancel Next >



The screenshot shows the 'Hardware Setup' window with the 'Install Third-Party Software (Step 1 of 1)' tab. It contains instructions on using the Pixy2 Vision Sensor Block and a table with download links for PixyMon v2. On the right, there is explanatory text under 'About Your Selection'. At the bottom right, the 'Next >' button is circled in red.

Hardware Setup

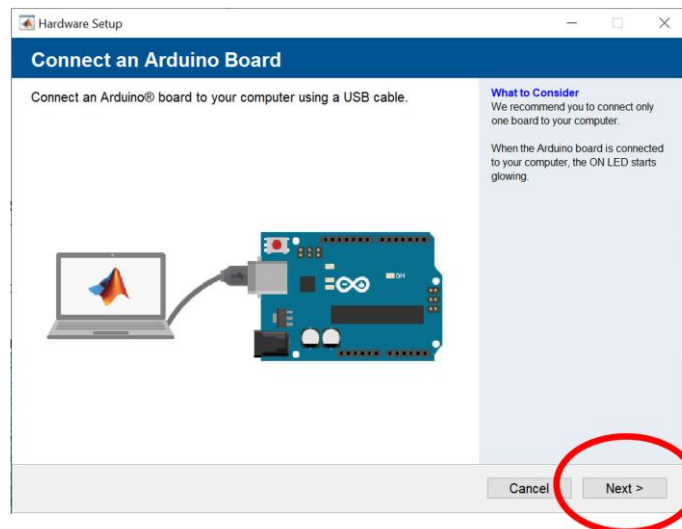
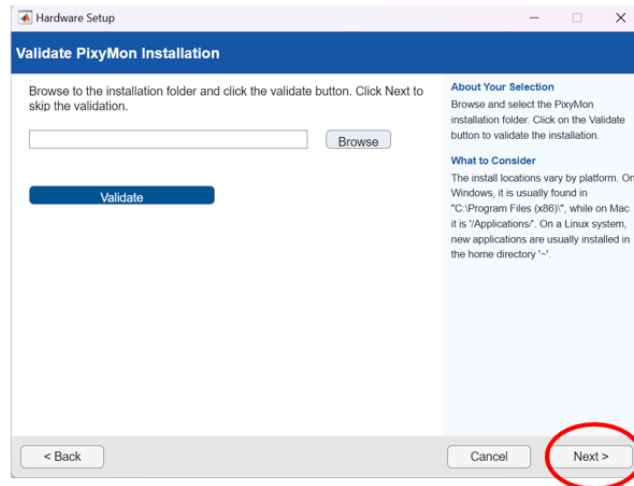
Install Third-Party Software (Step 1 of 1)

Use of the Pixy2 Vision Sensor Block in the Simulink® Support Package for Arduino Hardware requires the following software. Please go to the link in the table below and download the software. Refer the 'installation docs' links provided in the page for operating system specific instructions.

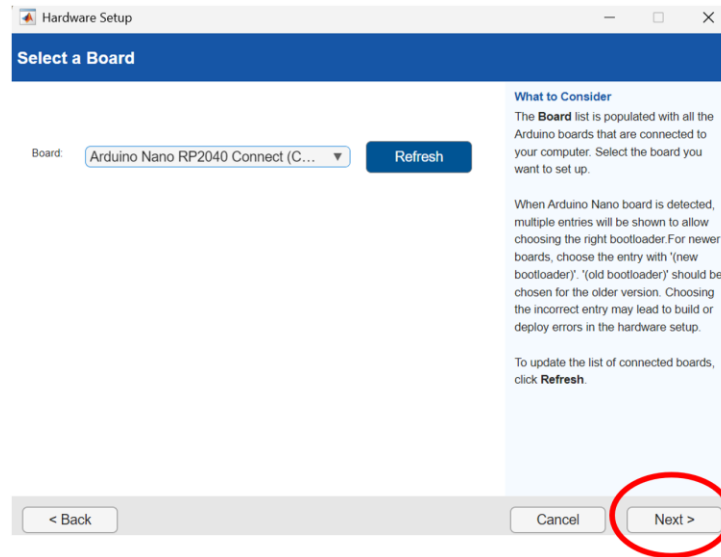
Name	Download Link
PixyMon v2	PixyMon v2 Download Page

About Your Selection
PixyMon v2 is the configuration utility for the Pixy2 Vision Sensor that runs on Windows, MacOS and Linux. It is required for training the sensor for object detection and is also useful for debugging.

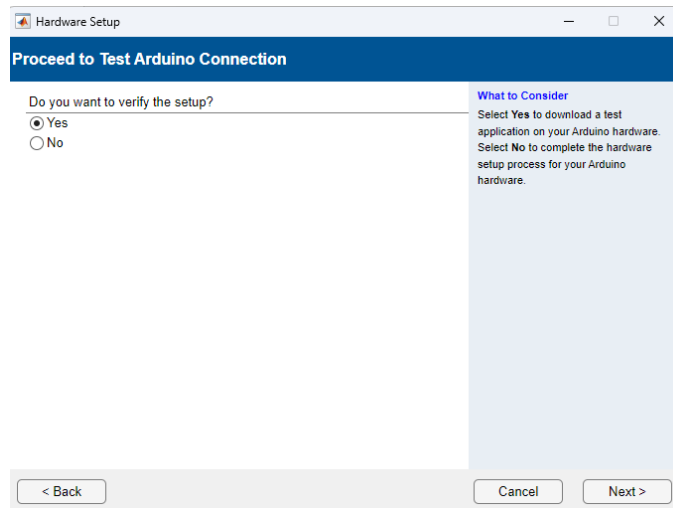
< Back Cancel Next >



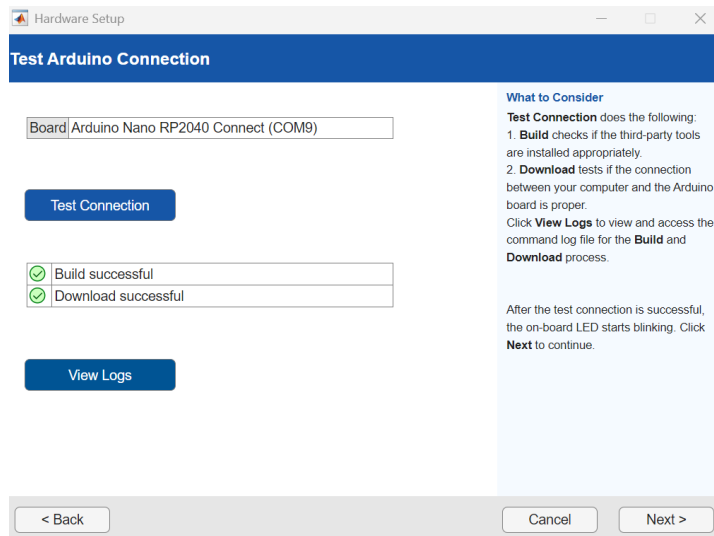
Make sure your board is plugged in and select Next. It will take a couple minutes before getting to the next screen. The next screen should automatically detect your board in which case you can select "Next".



Click Next to verify the setup.



Click "Test Connection". Verify it builds and downloads:



You are now ready to use your hardware.

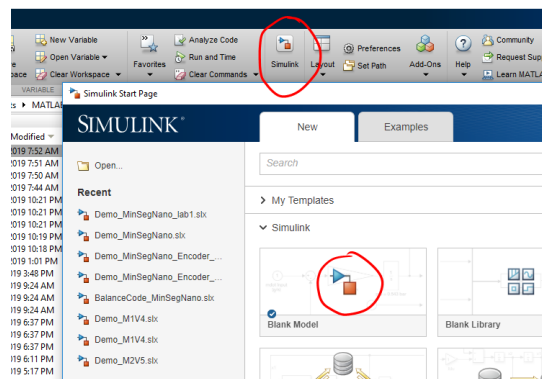
Part 2: Communicating with External Mode

Objective

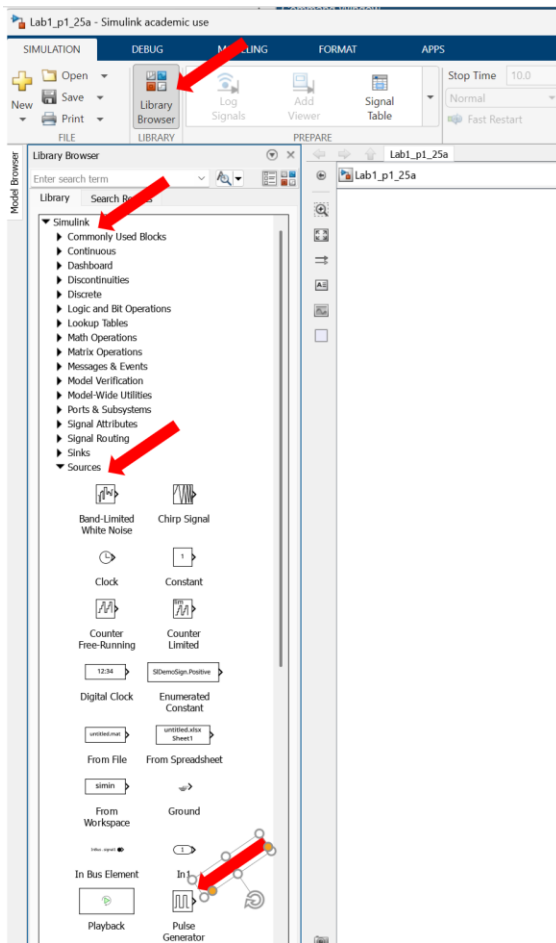
- Communicate with the target board (Arduino) using external mode (Monitor to turn LED 13 on and off).

Simulink

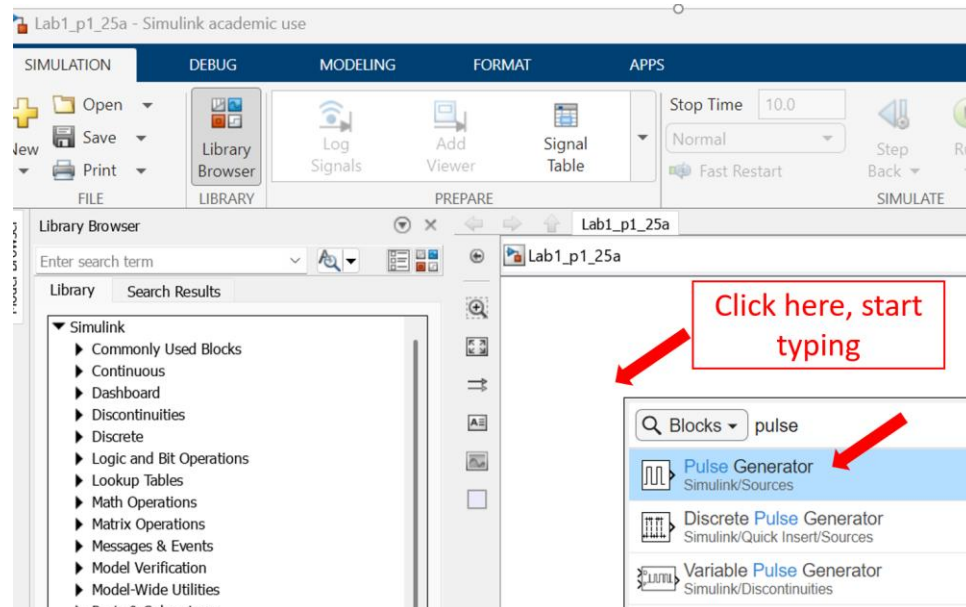
- 1) Open MATLAB then open a new Simulink model



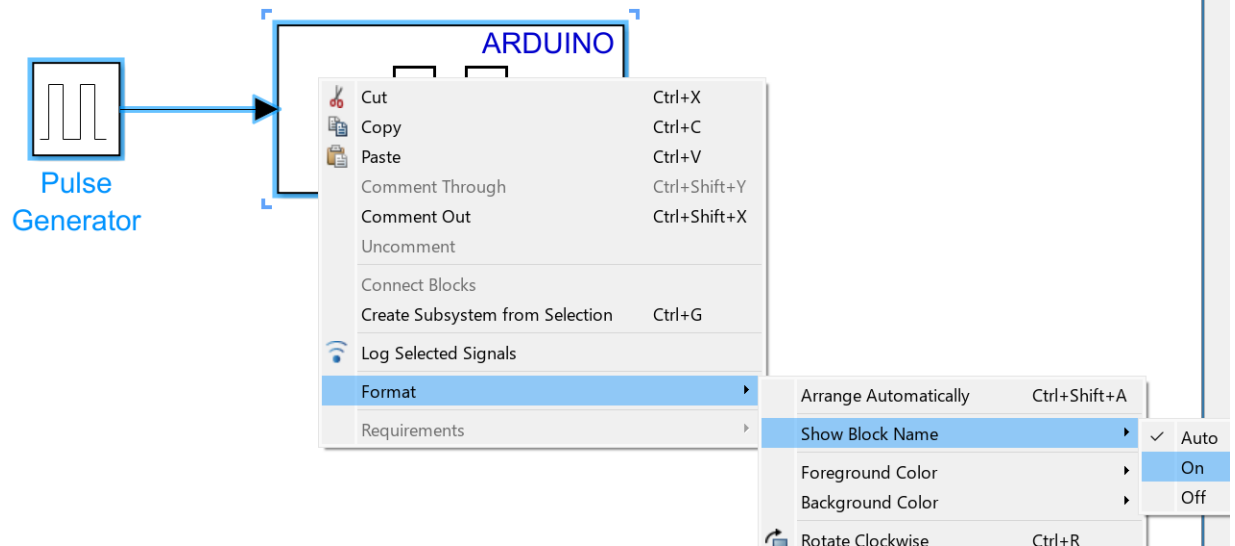
- 2) Find and assemble the required blocks.
 - Use the Library Browser under Simulink → Sources



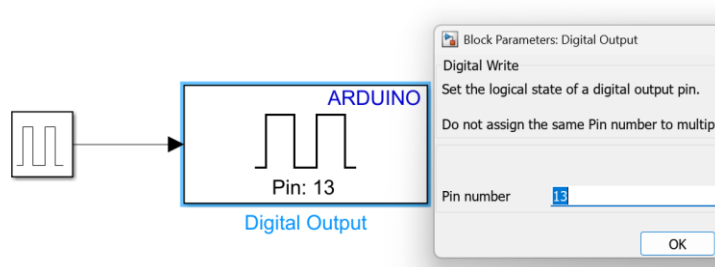
Alternatively you can just click anywhere in the blank area of your model and start typing the name of the block. Click in the blank area and start typing “pulse generator”



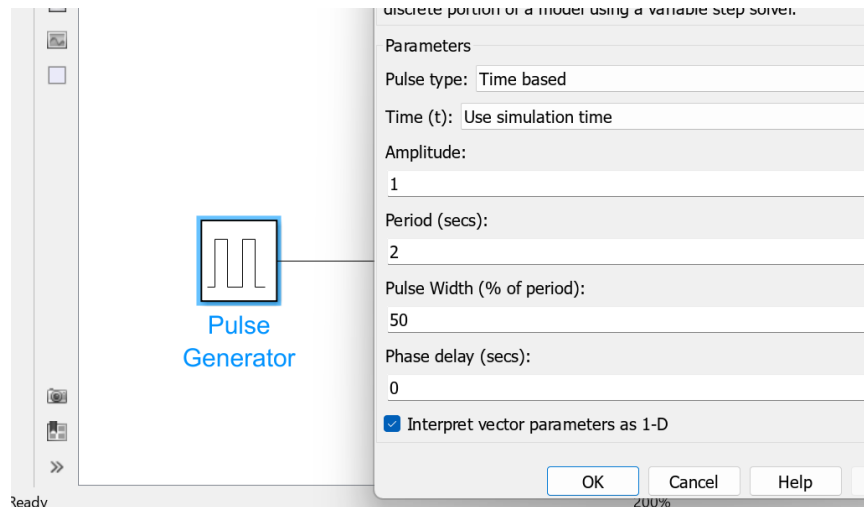
- Digital Output is under View->Library Browser->Simulink Support Package for Arduino Hardware->Common. Or start typing “Digital Output” in a blank space.
- Notice that by default Simulink does not show the block name. To show the name select all your blocks, right-click, “Format”, “Show Block Name”, “On”:



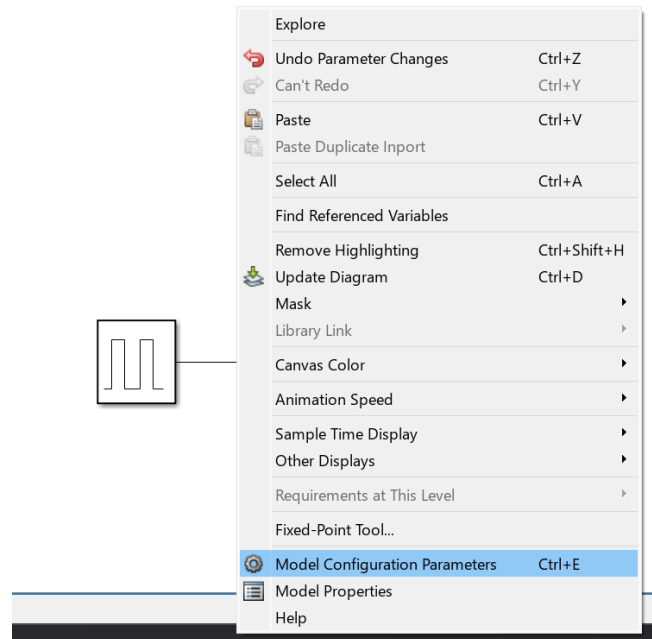
- 3) Change the output pin to 13, which is connected to the onboard LED. This is done by double clicking on the Digital Output block.



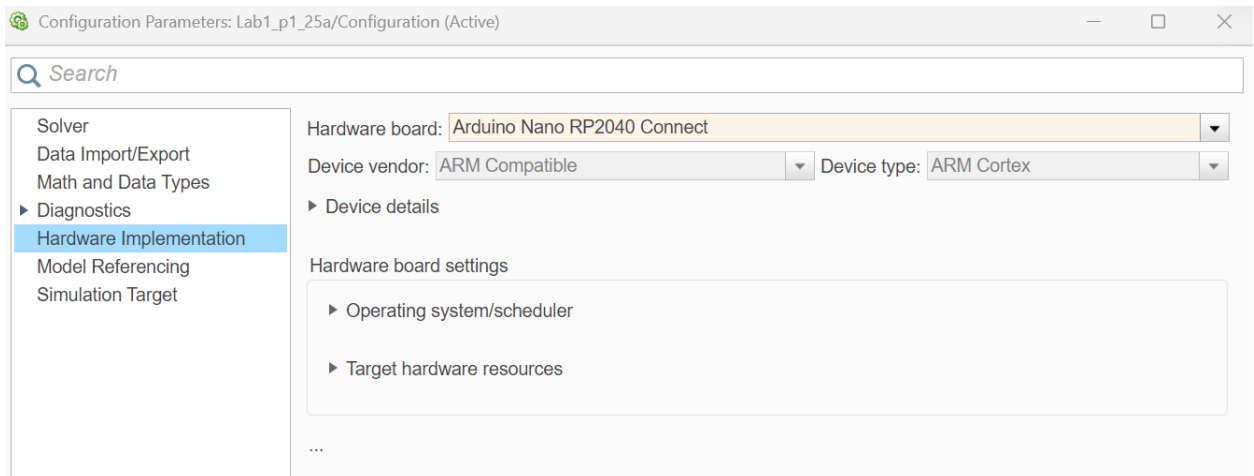
- 4) Double click on the Pulse Generator to change the amplitude to 1, the Period to 2, and the pulse width to 50%.



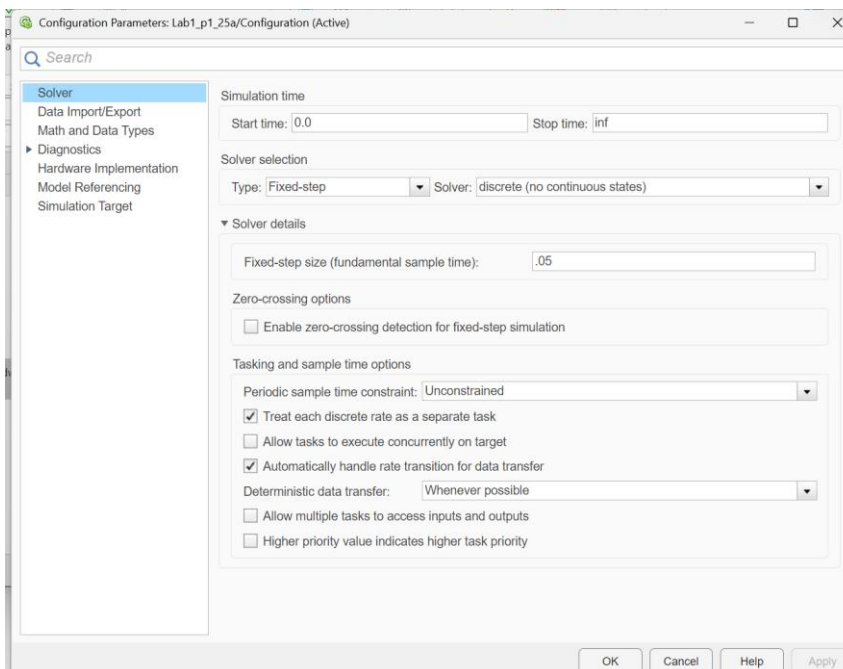
- Right click on the background of the Simulink window and select 'Model Configuration Parameters', or  in the top of the Simulink window. Or use the keyboard shortcut "Ctrl+E"



- Under Hardware Implementation, Hardware Board, Select Nano 3.0. This will set up your Simulink diagram to be used with hardware.



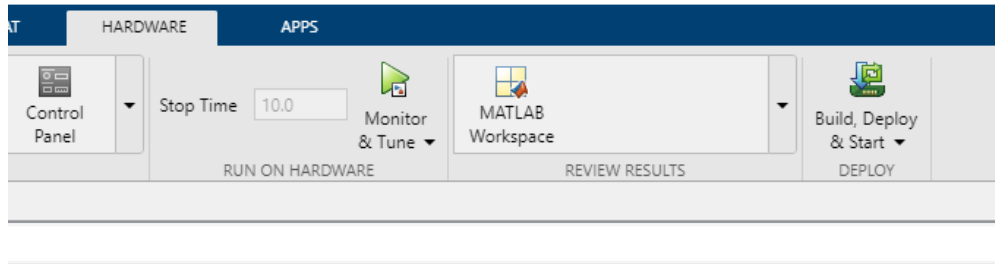
- Go to 'Solver' on the left hand menu and type 'inf' into the 'Stop time' spot so that the model will keep running. Expand "Solver details" and make sure the solver options are set to "Fixed-step" and "discrete (no continuous states)". The "Fixed-step size" should be 0.05. This is how fast in seconds the control loop will execute. In this case, every 50 milliseconds it will execute the Simulink code.



Deploy to Hardware (Normal Mode):

Compile and download the code to the board:

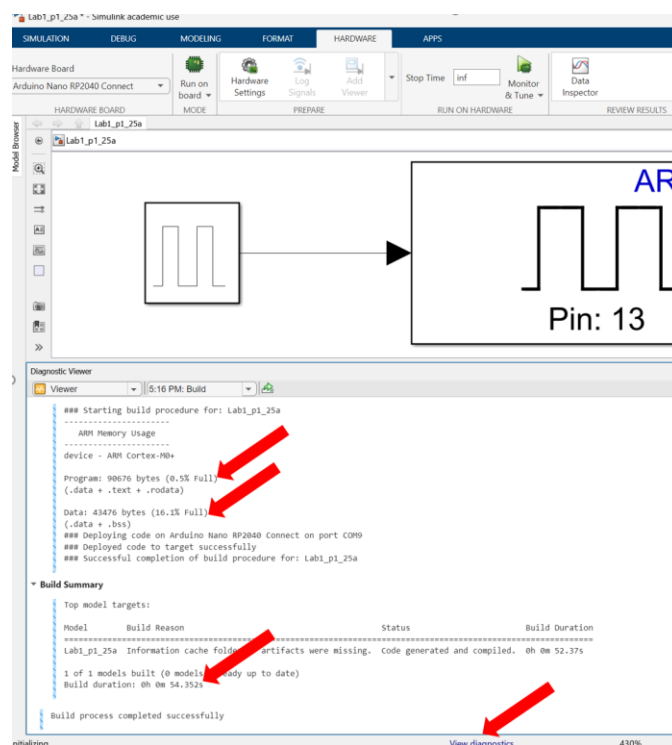
- Under the hardware tab select the Build, Deploy and Start button



This will download the code to the board and **will not interact or connect with it**. When “Deploy to Hardware” is used to download to the board no information is passed between it and your computer unless your program specifically instructs the board to do so.

At this point the onboard LED connected to pin 13 should start blinking on and off at every second.

You can see how long the compilation process takes as well as how much memory is used by clicking on the “View diagnostic” text in the bottom of the simulation diagram:

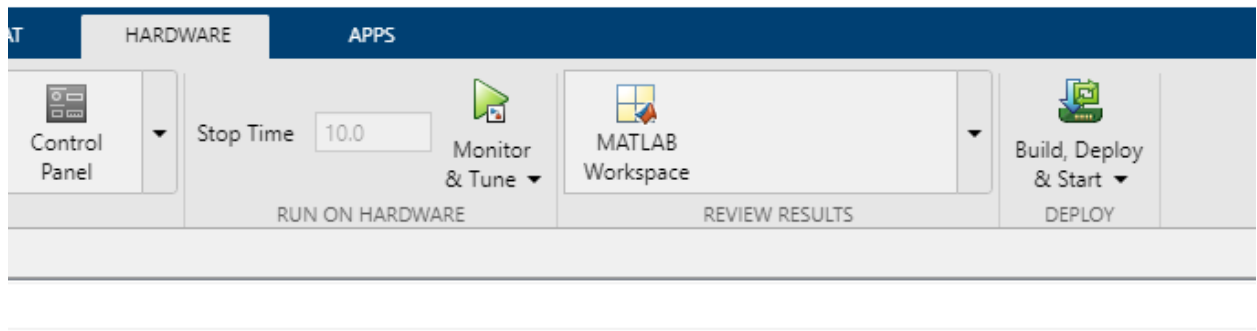


Monitor & Tune (External Mode) – communicating with the hardware

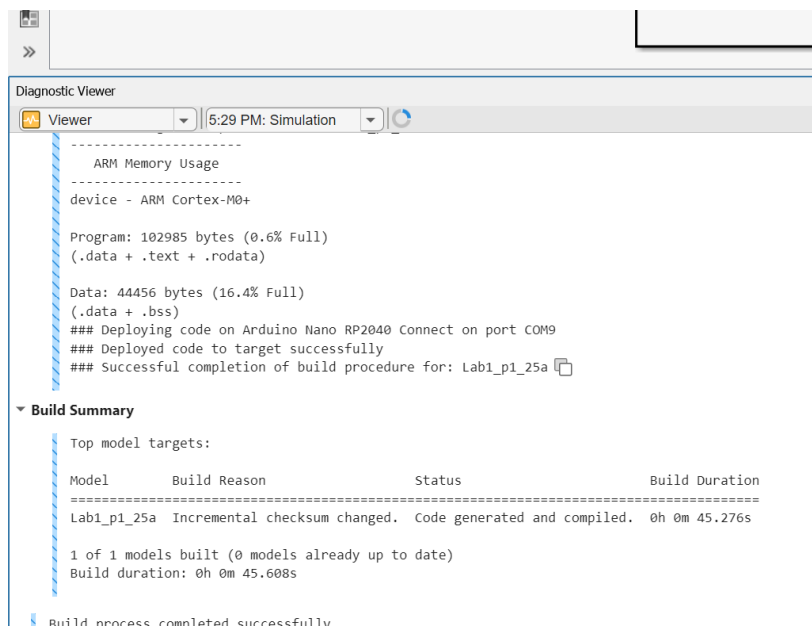
Monitor and Tune also called External mode will establish a data connection with the system so you can view data or interactively change parameters and see the result on the hardware in real time.

Monitor and Tune (External Mode):

- 1.) External mode enabled by pressing the green “Monitor & Tune” button to download the code. When the green play button is used the hardware will create a connection with your computer to exchange data. This will allow you to modify parameters in your model and see the results on the hardware:

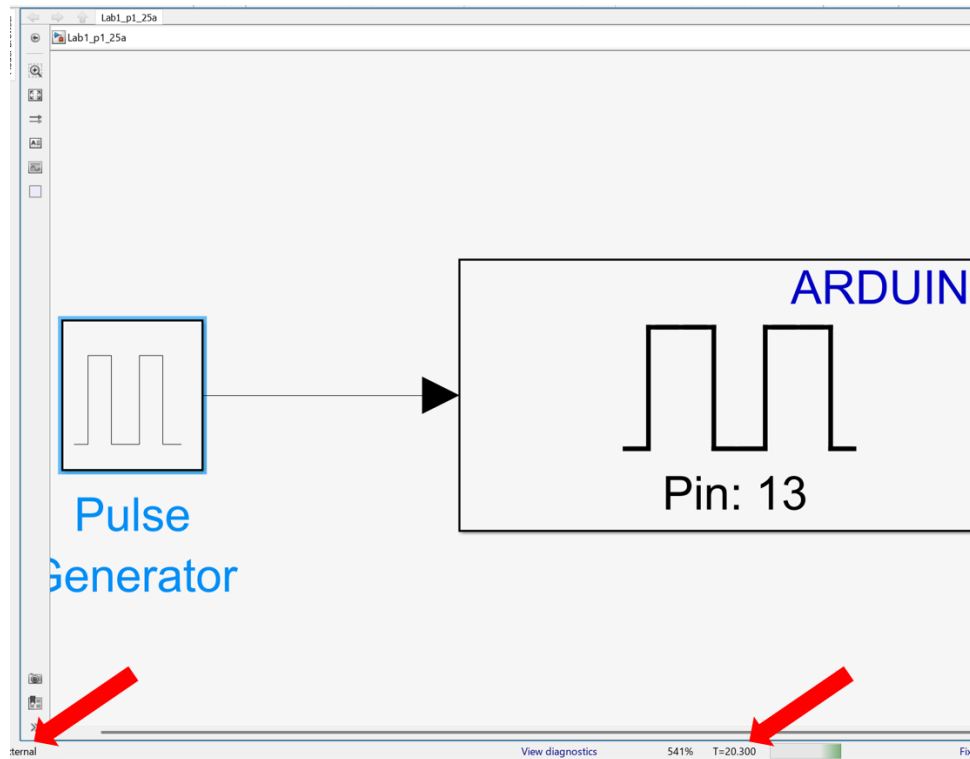


Notice as it is compiles how much larger the program size is:



Notice that to allow communication with the board it uses a slightly larger amount of memory. On some systems such as the original Arduino nano this extra memory can be very significant to when the overall memory is small - the cost over interactive communication can be a challenge for larger programs. This is why an Nano RP2040 (Raspberry Pi 2040 processor) is nice to use – it is low cost with large memory and good speed.

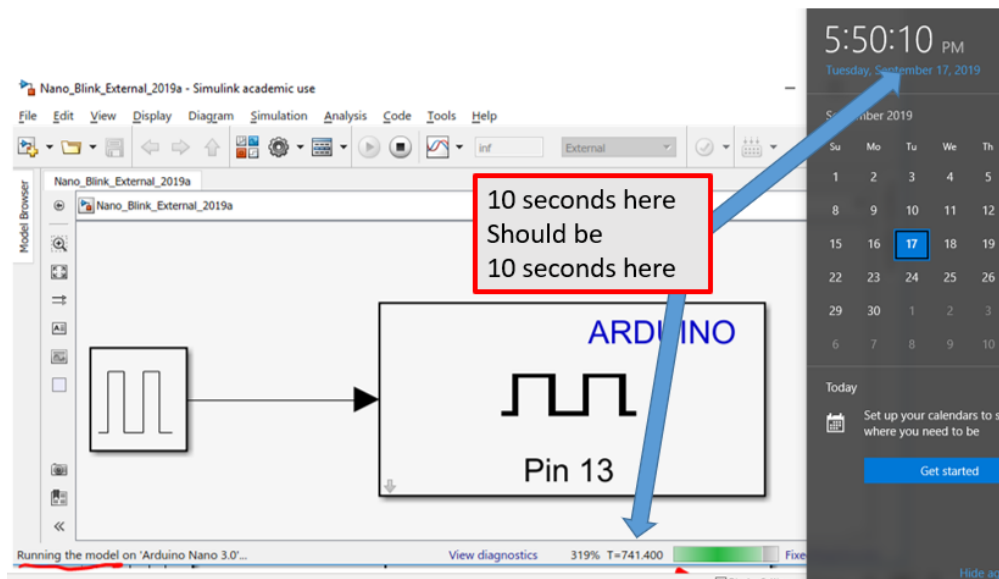
After the code is downloaded it will connect to the hardware and you will see the time progressing in the bottom right corner:



- 2.) Change parameters in your diagram and observe changes on the hardware. Double Click the Pulse Generator block and change the Period to 0.1 seconds. You should notice that the LED will begin blinking much faster.
- 3.) Use this file for new projects! For new hardware projects always use this file – it is configured to work with the hardware and you will not need to perform all the configuration steps again.

External Mode (Monitor and Tune)

- **External mode** and the green play button **allows bi-directional communication** to/from the application board to the PC, but can limit how fast your code can execute.
 - The “**Deploy to Hardware Button**” downloads the code and **does not connect or interact with it**, it can allow your code to run faster. You can use the “**Deploy to Hardware Button**” to download the code and NOT communicate to it.
- When external mode is selected and the green play button is pressed it downloads additional code to the board to allow this communication. This **increase** the program size (requires more memory)
- When you use external mode, you can change parameters while the system is running
 - However, this has an impact on the performance due to the communication bandwidth.
 - You can tell how fast you can run your code by examining the running clock in the Simulink diagram after you press the play button. When the code is running the green bar will be flashing and the time in seconds should be advancing:



If this clock advances as fast as the normal real world clock your code is “keeping up”. If, for example, it can take 1 minute for the this timer to reach 10 seconds your code is not keeping up with real time and your requested sample time is too fast – it can not compute everything it needs to do fast enough.

Stopping the Simulation:

When you are running the simulation in external mode you are connected to the device and are sending data and receiving data.

- Use the green “Play” button to download and run the code in external mode
- Use the square “Stop” button to stop.

Checkpoint:

To complete Lab 1 you will need to show the ability to control the blink of the LED by changing the parameters of the pulse block in external mode.

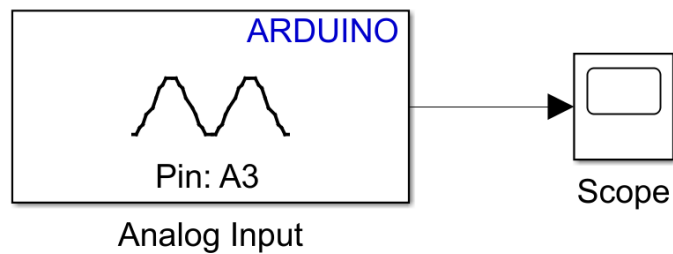
Part 3 (Optional): Evaluating code execution speed – how fast can I run my code

Objective:

- Determine how fast your code can be run on the target hardware using “Enable overrun detection” in Simulink.

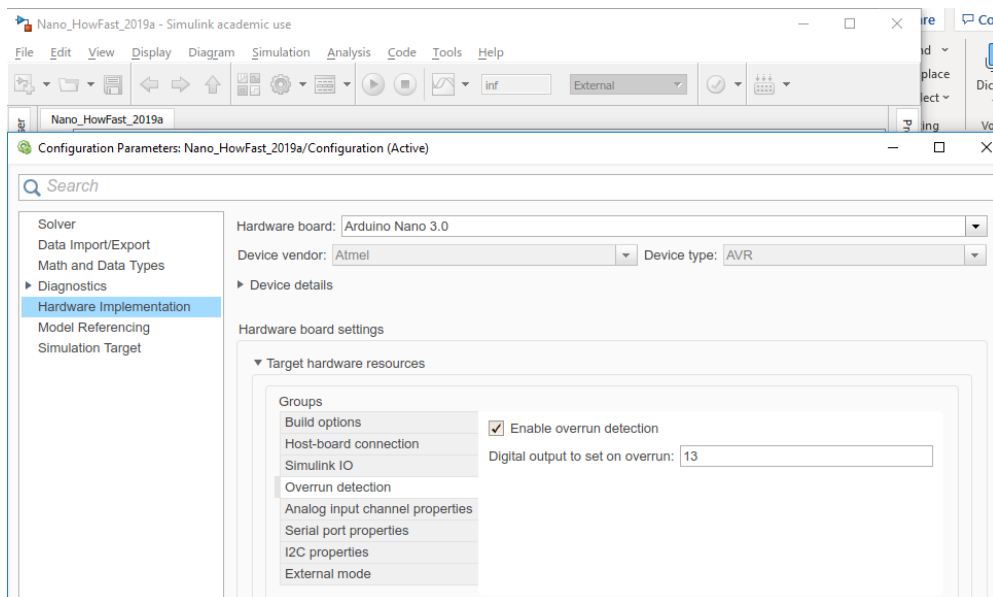
Simulink:

Build the following model:



This code simply reads a single analog channel. The terminator is also in the commonly used blocks'. To determine how fast the hardware can execute a simple loop check the box "Enable overrun detection" and set the pin to 13.

In the "Model Configuration Parameters" under "Hardware Implementation", "Target hardware resources", "Overrun detection" make sure "enable overrun detection is checked" and the output pin is set to 13.



If the hardware cannot execute the code in the time specified the digital output, in this case the LED on pin 13 will light up indicating that the code cannot be executed in the time you have specified. Any digital output can be used, but pin 13 is conveniently connected to an LED on the board. For the example code above, start with a sample time of 0.1 milliseconds (.0001 seconds). Download the code with the deploy to hardware button (Deploy to hardware mode, NOT external mode play button) and

observe if LED 13 on the board lights up after downloading:

Configuration Parameters: Lab1_p2_25a/Configuration (Active)

Search

Solver

Data Import/Export

Math and Data Types

► Diagnostics

Hardware Implementation

Model Referencing

Simulation Target

Simulation time

Start time: 0.0 Stop time: inf

Solver selection

Type: Fixed-step Solver: discrete (no continuous states)

▼ Solver details

Fixed-step size (fundamental sample time): .0001

Zero-crossing options

☐ Enable zero-crossing detection for fixed-step simulation

Tasking and sample time options

Periodic sample time constraint: Unconstrained

☒ Treat each discrete rate as a separate task

☐ Allow tasks to execute concurrently on target

☒ Automatically handle rate transition for data transfer

Deterministic data transfer: Whenever possible

☐ Allow multiple tasks to access inputs and outputs

☐ Higher priority value indicates higher task priority

OK Cancel Help Apply

You should notice LED 13 will light up. This indicates the microprocessor cannot run the code every 0.0001 second, so you have to specify a larger sample time

Keep increasing the sample time (try .2 milliseconds, etc.) until LED 13 does not light up after downloading. This is the fastest you the board can execute your code. How fast can you run this code using Deploy?

In this fashion, you can determine how fast your code, or any part of your code takes to run on your hardware.

You can also use this test with Monitor and Tune. Try using Monitor and Tune see what is the fastest sample time you can have and still read the scope for 10 seconds (it might take a couple seconds for the system to overflow)?